A Simple and Efficient Heuristic Algorithm For Maximum Clique Problem

Krishna Kumar Singh
Dept of CSE, RGUKT-IIIT Nuzvid (AP), India
krishnasingh@rgukt.in

Lakkaraju Govinda
Dept of CSE, RGUKT-IIIT Nuzvid (AP), India
govinda.891@gmail.com

Abstract: **A clique is a sub graph in which all pairs of vertices are mutually adjacent. A maximum clique is a maximum collection of objects which are mutually related in some specified criterion. This paper proposes an efficient heuristic approach for finding maximum clique using minimal independent set in a graph. At each recursive step, the algorithm finds minimal independent vertices for further expansion to get adjacent list, reason is that at the depth, the maximum clique most likely would include either of the vertices of minimal independent set. Further a pruning strategy is used to abort smaller size of clique to be explored.**

Keywords: **Maximum clique, Heuristics, Minimal independent set**

I. INTRODUCTION

Given a graph G(V, E), where V is the set of vertices and E is the set of edges, d(v) is degree of vertex v□V, Adj(v) is a set of neighbors of vertex v. A complete sub graph of G is one whose vertices are pair wise adjacent. A clique is a complete sub graph S of a given graph G, where all vertices presented in S are mutually adjacent to each other. Finding a clique of a fixed size k is a well known and deeply studied NP-complete problem known as k-clique [1].The corresponding optimization problem i.e. finding the maximum complete sub graph of G is known as the Maximum Clique Problem (MCP) [2]. MCP finds applications in many fields; Bioinformatics, coding theory, fault diagnosis, pattern recognition, and computational biology [3, 4, 5, 6, 7, 8], computer vision [9], robotics [10] etc. Therefore, it is required to develop exact maximum-clique finding algorithms that run very fast in practice. Since it is NP-Hard, it is difficult to obtain a polynomial time algorithm to find exact solution efficiently [2], but using some heuristics and pruning condition, the performance of algorithm may be obtained considerable. An independent set is a set of vertices whose elements are pair wise nonadjacent. The minimal independent set is found using greedy approach, i.e. finding non adjacent vertex one by one with maximum degree first. Like other algorithms to get maximum clique, this also follows branch and bound technique. Algorithms for the maximum clique problem have been studied by many authors (see references [11, 12, 13, 14]. There all are based on branch-and- bound, and backtracking techniques with some pruning conditions.

This paper proposes a simple and very efficient algorithm for finding the maximum clique in a graph. The algorithm does enumeration for an independent set of nodes on each recursive step. Computational comparison with various DIMACS benchmark graphs shows that the algorithm gives better performance than any of the previous well known methods. The reason of better performance of the proposed algorithm is that, the search space gets drastically decreased because at each recursive step only non adjacent vertices are likely to be explored. Second reason is that, for high edge density (d>0.5) graphs, the minimal independent vertices found (at each recursive step) in the range given by the Equation (1), are most likely to include the maximum clique.

The proposed algorithm is discussed in Section 3. Comparisons of computational time for various DIMACS benchmark graphs are discussed in Section 4. The paper is concluded in Section 5.

II.RELATED WORKS

Several papers are published since 1975s. Few of the most significant papers are referred here. The easiest and effective one was presented by Carraghan and Pardalos [11]. The algorithm uses a partial enumeration of vertices. It is generally used as a basis for computational comparison for afterward almost all proposed algorithms. Another algorithm was published by P. Östergård [12] used a different ordering of vertices and asserted that the ordering of vertices have effect on computational time for different types of graph. He also has compared his algorithm with earlier published algorithms and has shown his algorithm works better. P. Ostergård algorithm is just modifications of Carraghan and Pardalos' algorithm. One of the recent algorithm [13] presented by Deniss Kumlander, based on a fact that vertices from the same independent set couldn't be included into the same maximum clique. Those independent sets are obtained

978 -1- 4799 - 3837 - 7/14/$31.00© 2014 IEEE

from a heuristic of vertex coloring where each of the independent set is a color class.

III. Proposed Algorithm

The algorithm proposed is based on a heuristic that at each recursive step only the vertices of minimal independent set at the depth are explored further to get adjacent list(from the current list). In other word, at each recursive step, the algorithm finds minimal independent vertices for further expansion; reason is that at the depth, the maximum clique most likely would include either of the vertices of minimal independent set.

Initially, the algorithm considers an ordering of the vertices of G, say v1, v2, … vn, where d(v1)≥d(v2)≥…d(vn). It is observed that this vertex ordering may reduce the computational time in problems with dense graphs. Let vi, vi+1,…vi+k are non adjacent vertices (minimal independent set) at depth d. Initially, the algorithm finds the largest clique C1 that contains the vertex vi, then it finds C2; the largest clique in G-{vi) that contains vi+1 and so on. There is a notion of the depth. Suppose we consider vertex vi at depth d, it considers all vertices (that are in depth d and) adjacent to vi are at depth d+1, and so on. Let vi is expanding currently, if neighbors of vi (Adj(vi)) in current list plus the current depth < size of current best clique (CBC), then it is pruned, since the size of the largest possible clique (formed by expanding vi) would be less or equal to the size of CBC. If currently expanding list size is one then also it is pruned; this is how the maximum clique is found. Moreover, from experimental result it is observed that for high edge density (d>0.5) graphs, the minimal independent vertices found (at each recursive step) in the range given by the Equation (1), are most likely to include the maximum clique.

$$\lfloor d|U| \rfloor \leq vi \in V \leq \lceil (d - 0.3)|U| \rceil \dots\dots\dots (1)$$

The first vertex of current list is selected as the first vertex of independent set, and the remaining non- adjacent vertices are selected from the range as per the equation (1).

Proposed Algorithm

```
function clique(U, depth)
1:  if |U|=1 then
2:          if depth>max then
3:              max =depth
6:      return
7:  V=Minimal_Independent_Set(U)
8:  for i in 1 to |V|
9.      vi= U ∩ N(Vi)   //vi is a set of adjacent vertices of Vi in U
11.     if depth + |vi| ≤ max then
12:             return
13:       clique(vi, depth + 1)
14:   return
15:function main
16:      max:=0
17:      clique(V, 1)
18:     return
```

It follows the similar strategies as discussed in existed algorithm [11] in addition with finding minimal independent set at each recursive step, but there is not any maintenance of ordering at each recursive step.

IV. Example

The algorithm is described by taking one example below.

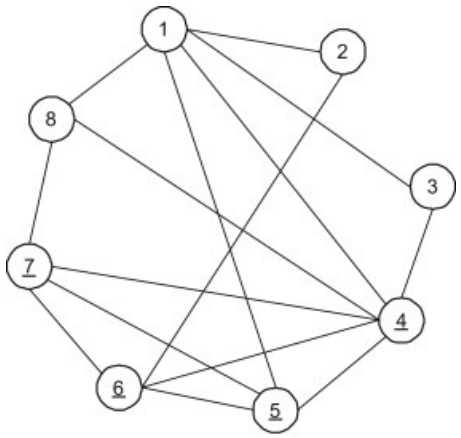

Figure 1: A simple graph with clique size 4

Vertices are initially sorted (as per maximum degree first): 4 1 5 6 7 8 2 3

Initially max (Clique)=1

Only the conditions which becomes true (causes its body to be executed) are mentioned in each step (Depth)

Depth 1: U={ 4 1 5 6 7 8 2 3}, max=0
(Independent vertices) V= { 4 2}
Adj(4)= {1 5 6 7 8 3 }
Adj(2)= {1 6 }
Depth 2: U= {1 5 6 7 8 3}, max=0
V= {1 6}
Adj(1)= { 5 8 3 }
Adj(6)= { 5 7 }
Depth 3: U= {5 8 3}, max=0
V= {5 8 3}
Adj(5): ϕ
Adj(8): ϕ
Adj(3): ϕ
(Depth > max) true, so max=3
return
Depth 3: U= {5 7} , max=3
V= {5}
Adj(5)= { 7 }
Depth 4: U= {7 }
|U|=1 true,
Depth > max true, Max=4
return
Depth 2: U={1 6}
Depth+|U|≤max, true
return
Output: max (Clique) =4 (i.e. 4, 6, 5, 7)

V. COMPUTATIONAL PERFORMANCE

A comparative computational performance is enlisted in Table-1. The execution time is taken in micro seconds and they are executed on system with configuration; Intel(R) Core (TM) -i3 @2.3 GHz. It is observed that the algorithm presented by Deniss Kumlander [13] is better than the algorithms presented in [11] and [12]. The proposed one (MISBA; Minimal Independent Set based Approach) is better than the algorithm presented in [13], either in term of computational time or in term of maximum clique size. The last column value in the Table-1 is obtained by the proposed (MISB) algorithm. The integer (in the Table-1) written before slash is obtained maximal clique size by the corresponding (column heading) algorithm, and integer written after slash or without slash indicates execution time in microsecond. Execution of the algorithm gives exact output for most of the bench mark graphs e.g. C-fat-200, C-fat-500, Johnson, Hamming, P-hat-300, Keller, (with vertices less or equal to 500, considering the constraints of computer system available). For other bench mark graph like P-hat500, brock, gen, sanr, rand400, it gives lesser (but close) size of clique, but performance is better than earlier well known approaches referred in [11][12][13].

TABLE-1: THE EXECUTION TIME (IN MICRO SECONDS) FOR VARIOUS BENCHMARK GRAPH

					Execution Time in micro second		
Benchmark Graphs	**#Vertices**	**# Edges**	**Edge Density**	**Max Clique**	**Dfmax[11]**	**VColor-BT-u [13]**	**MISBA (Proposed)**
c-fat200-1.clq	200	3235	0.16	12	2319	347	195
C-fat200-2.clq	200	3235	0.16	24	2588	5623	144
C-fat200-5.clq	200	8473	0.43	58	339236382	2876	191
C-fat500-1.clq	500	4459	0.04	14	6477	1067	530
C-fat500-2.clq	500	9139	0.07	26	8773	795	121
C-fat500-5.clq	500	23191	0.19	64	2431982	1565	146
C-fat500-10.clq	500	46627	0.37	>=126	> 2.5 hours	7481	231
Johnson16-2-4.clq	120	5460	0.76	8	927134	364	968
Johnson32-2-4.clq	28	210	0.56	4	264	131	39
Johnson8-4-4.clq	70	1855	0.77	14	14678	408	250
hamming6-2.clq	64	1824	0.9	32	22504	31/745	57
hamming6-4.clq	64	704	0.35	4	674	156	173
Hamming8-2.clq	256	31616	0.97	128	>2 hr	120/6513	225
hamming8-4.clq	256	20864	0.64	16	2469071	461	6785
brock2001_1.cq.b	200	14834	0.75	21	19230025	16/3291	19/5184
brock2001_2.cq.b	200	9876	0.5	12	47696	8/ 1117	6064
brock2001_3.cq.b	200	12048	0.61	15	295687	12/ 1618	1624
brock2001_4.cq.b	200	13089	0.66	17	1182613	13/ 2341	27216
brock4001_1.cq.b	400	59723	0.75	27	>8 hours	19/2966	23/76322
brock4001_2.cq.b	400	59786	0.75	29	>8 hours	19/2828	23/84414
brock4001_3.cq.b	400	59681	0.75	31	> 8 hours	20/2936	23/99845
brock4001_4.cq.b	400	59765	0.75	33	> 8 hours	19/2881	23/64783
keller4.clq.b	171	9435	0.65	11	484575	9/ 1138	1569
MANN_a9.clq	45	918	0.93	16	52370	286	603
MANN_a27.clq	378	70551	0.99	126	>5 hours	125/17249	125/328
p-hat300-1.clq	300	10933	0.24	8	17484	6/360	2956
P-hat300-2.clq	300	21928	0.49	25	842165	19/ 1012	129239
P-hat300-3.clq	300	33390	0.74	36	1066206601	27/2088	34/51618
P-hat500-1.clq	500	31569	0.25	9	78842	7/874	6185
P-hat500-2.clq	500	62946	0.5	36	181509952	29/2942	34/43216
P-hat500-3.clq	500	93800	0.75	>=49	>2 hours	39/5947	47/1120684
gen200_p0.9_44.b	200	17910	0.9	44	>2hours	35/ 2008	39/352999
gen200_p0.9_55.b	200	17910	0.9	55	>2 hr	31/2482	49/62352
sanr200_0.7.clq	200	13868	0.7	18	4026395	14/1202	16/6366
Sanr200_0.9.clq	200	17863	0.9	>=42	> 2 hours	32/2022	39/868502
sanr400_0.5.clq	400	39984	0.5	13	13/2883467	9/1196	12/34088

VI. Conclusion

This paper proposes an efficient heuristic approach for finding maximum clique using minimal independent set in a graph. The algorithm is efficient because the search space gets drastically decreased, as at each recursive step only non adjacent vertices are likely to be explored. Second reason is that, for high edge density (d>0.5) graphs, the minimal independent vertices found (at each recursive step) in the range given by the Equation (1), are most likely to include the maximum clique.

VII. References

[1] Karp, R.M: In: Miller, R.E., Thatcher, J.W. (eds.) Reducibility among Combinatorial Problems, pp. 85–103. Plenum, New York (1972).

[2] Bomze, I.M., Budinich, M., Pardalos, P.M., Pelillo, M.: HandBook of Combinatorial Optimization. Supplement A. pp. 1–74. Kluwer Academic Publishers, Dordrecht (1999).

[3] Bahadur, D.K.C., Akutsu, T., Tomita, E., Seki, T., Fujijama, A.: Point matching under non-uniform distortions and protein side chain packing based on efficient maximum clique algorithms. Genome Inform. 13, 143–152 (2006)

[4] Butenko, S., Wilhelm, W.E.: Clique-detection models in computational biochemistry and genomics.Eur.J.Operat.Res.173, 1–17 (2006).

[5] C. Hofbauer, H. Lohninger, A. Asz´odi, SURFCOMP: A novel graph based approach to molecular surface comparison, Journal of Chemical Information and Computer Science 44 (2004) 837-847.

[6] S. Schmitt, D. Kuhn, G. Klebe, A new method to detect related function among proteins independent of sequence and fold homology, Journal of Molecular Biology 323 (2002) 387-406.

[7] J. Konc, D. Janeˇziˇc, A branch and bound algorithm for matching protein structures, Lecture Notes in Computer Science 4432 (2007) 399-406

[8] J. Konc, D. Janeˇziˇc, Protein-protein binding-sites prediction by protein surface structure conservation, Journal of Chemical Information and Modeling 47 (2007) 940-944.[9]. Hotta, K., Tomita, E., Takahashi, H.: A view invariant human FACE detection method based on maximum cliques. Trans. IPSJ 44(SIG14(TOM9)), 57–70 (2003).

[9] San Segundo, P., Rodríguez-Losada, D., Matía, F., Galán, R.: Fast exact feature based data correspondence search with an efficient bit-parallel MCP solver. Appl. Intel. 32(3), 311–329 (2010).

[10] R. Carraghan, P.M. Pardalos, An exact algorithm for the maximum clique problem, Oper. Research. Letters. Vol. 9 (1990), pp 375–382.

[11] Östergård, P.R.J., A fast algorithm for the maximum clique problem. Volume 120, Issues 1–3, 15 August 2002, pp 197–207.

[12] Deniss Kumlander, A simple and efficient algorithm for the maximum clique finding reusing a Heuristic vertex colouring. IADIS international journal on computer science and information systems, Vol. 1, No. 2, 2006, pp. 32-49, ISSN: 1646-3692.

[13] D. Kumlander, A new exact algorithm for the maximum-weight clique problem based on a heuristic vertex-coloring and a backtrack search, Proceedings of The Forth International Conference on Engineering Computational Technology, Civil-Comp Press, 2004, p. 137-138.

[14] Ashay Dharwadker, The Maximum Clique problem, published by Amazon in 2011.